COMP532 - Machine Learning and Bio-inspired Algorithms: Assignment I
Bandit Arm Problem

March 4, 2026

Zhiheng Wang - 201865071
Laura Sierra Pena - l.v.sierra-pena@liverpool.ac.uk - 201936067
Ainur Smailova - 201874285
Yan-Yu Lin - 201948160
Joe Davies - 201950629
Pablo Ramos - sgpramos@liverpool.ac.uk- 201885602

Introduction

Reinforcement learning is an artificial intelligence approach using an intelligence agent which takes actions in a dynamics environment and learns from its interactions. The objective is to maximise the expected cumulative reward. For this assignment, we were asked to implement a multi-armed bandit algorithm. A multi-armed bandit problem is theoretically when in a gambling scenario with multiple slot machines, known as bandits, and playing to determine which machine will return the maximised reward. Using reinforcement learning, an agent has multiple arms as options to choose from, with the potential of providing a reward under uncertainty. Sequentially, actions will influence the agents' decisions by providing evaluative feedback.
Multi-armed bandits (MABs) are a simplified reinforcement learning setting where an agent repeatedly chooses among several arms to maximise reward under uncertainty. A central difficulty is the exploration exploitation trade-off. When action value estimates are maintained, the arm with the highest current estimate is the greedy choice. Selecting it is exploitation because it uses the best option according to current knowledge, while selecting a non-greedy arm is exploration because it can reduce uncertainty and improve future estimates. Exploitation tends to maximise expected reward in the next step, but exploration can lead to higher total reward over a longer horizon by helping the agent discover arms that are actually better. Since a single action choice cannot do both at once, balancing exploration and exploitation is unavoidable in bandit problems. (Sutton & Barto, 2018, Ch.2, Sec 2.1)
A common way to implement action value methods is to estimate each arm's value from observed rewards. One simple estimator is the sample average, computed as the total reward collected from an arm divided by the number of times it has been selected. (Antonacopoulos et al., 2025)
To compare bandit methods, performance is often summarised using reward based curves such as cumulative or average reward over time, together with the frequency of selecting the optimal arm. (Antonacopoulos et al., 2025)
In many real applications, reward distributions are non-stationary and can change over time. This motivates methods that adapt to recent observations rather than relying on full historical averages. Recent work has also explored explicit uncertainty measures, for example using the fractal dimension of reward history and integrating it into standard bandit algorithms, with

reported improvements in cumulative reward in non-stationary settings. (Nandhini & Pandian, 2025)

Methodology

Table 1: Agents and Environments

Estimator	Strategy	Environment
Total Average	epsilon = 0.1	Static (Stationary Bandits)
Total Average	epsilon = 0.3	Static (Stationary Bandits)
Total Average	epsilon = 0.5	Static (Stationary Bandits)
Total Average	epsilon = 0.7	Static (Stationary Bandits)
Total Average	epsilon = 0.9	Static (Stationary Bandits)
Total Average	softmax	Static (Stationary Bandits)
Total Average	softmax	Dynamic (Non-stationary Bandits)
Moving Average (window=100)	softmax	Dynamic (Non-stationary Bandits)
Moving Average (window=20)	softmax	Dynamic (Non-stationary Bandits)
Inc Update (step = 0.1)	softmax	Dynamic (Non-stationary Bandits)
Epsilon-Greedy (epsilon=0.1, step=0.1)	softmax	Dynamic (Non-stationary Bandits)
Inc Update (step = 0.3)	softmax	Dynamic (Non-stationary Bandits)

To approach this, we first compared types of agents with different strategies and update rules in different environments, specifically one static environment and one non-static (dynamic) environment, as seen in Table 1.
After analysing the different strategies, we decided object-oriented programming presented an efficient and effective way to implement the multi-armed bandit algorithm. We started by creating the base abstract arm class, with multiple subclasses, each with different distributions, secondly, a base abstract Estimator class which provides estimation of different actions and ways to update that estimation. Thirdly, a base abstract Strategy class, which selects which action to perform based on the estimations, and finally an Agent class, which is the combination of an estimator and a strategy, as well as recording the cumulative reward over time.
In our environment, we have a list of five arms, and ten agents. In each step, each agent selects an arms using its specific strategy. The agent receives a reward from the selected arm, and the agent updates its estimator with that reward. After that, the results are then plotted with cumulative reward in the y-axis and timesteps in the x-axis. The results show a range of cumulative rewards over timesteps, where a lower epsilon value corresponds to a higher cumulative reward.
To ensure reproducibility in between runs, we used NumPy's random seed module and selected different values for the seeds.

Results

0.1 Static Environment

In Figure 1 it can be observed that over many steps, small values for ε and softmax strategies accumulate reward much faster, leading to clearly higher curves. Strategies with large ε show significantly lower cumulative reward, which corresponds to higher cumulative regret.
The patterns match the best-arm plot: small ε (0.1, 0.3) and softmax achieve higher steady-state average reward. Large ε values suffer from excessive exploration, so their average reward stabilizes at a much lower level. This illustrates that, in a static bandit, too much exploration directly hurts long-run reward.

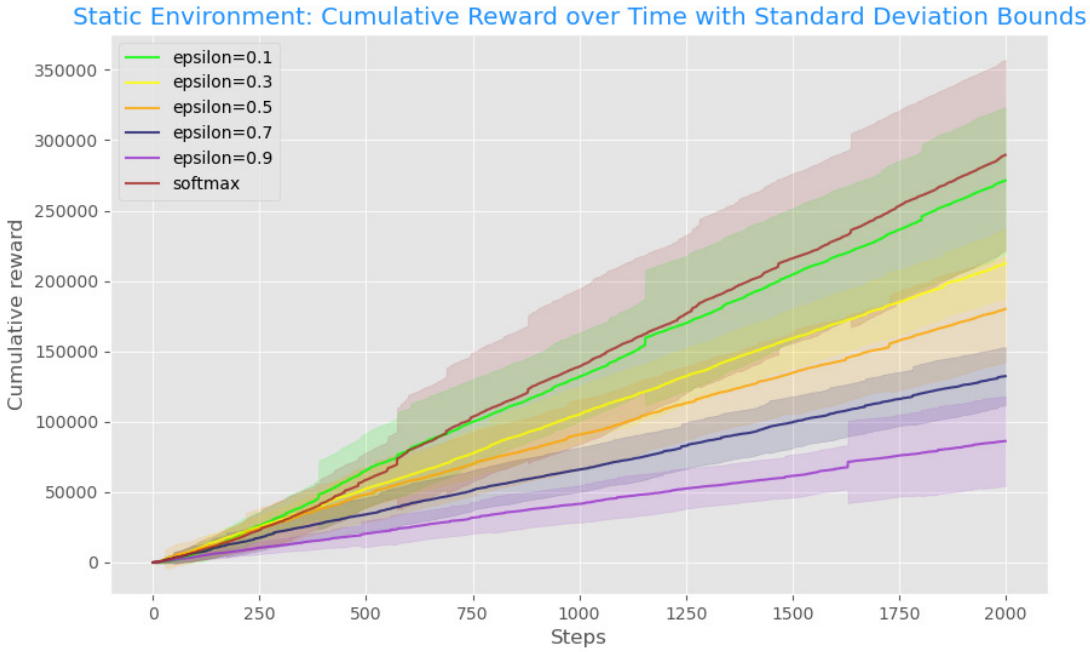

Figure 1: Cumulative reward over time with standard deviation bounds for static environment

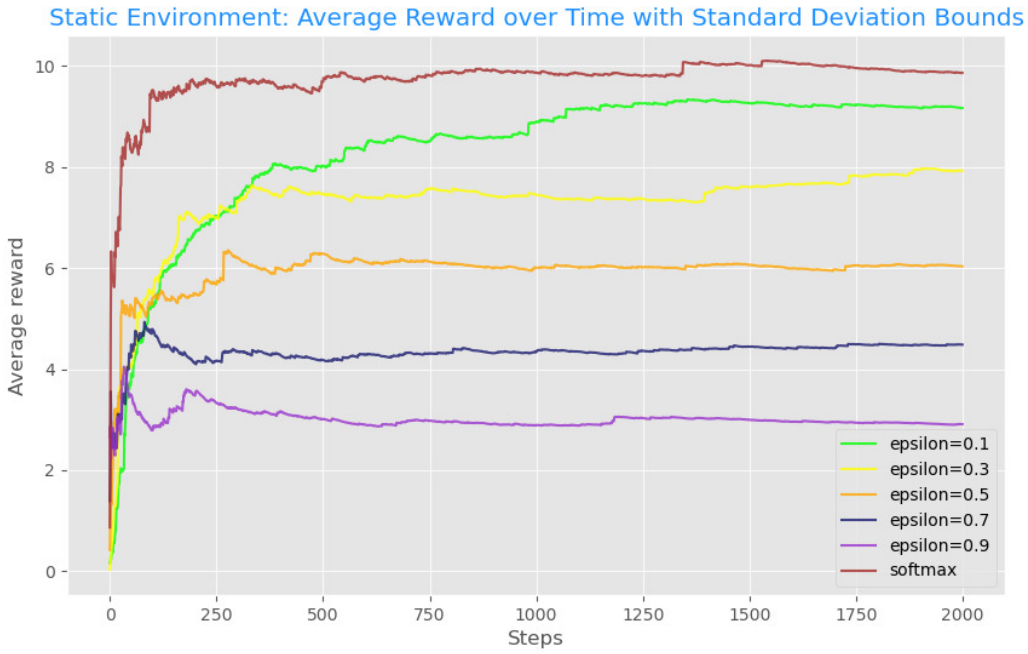

Figure 2: Average reward over time with standard deviation bounds for static environment.

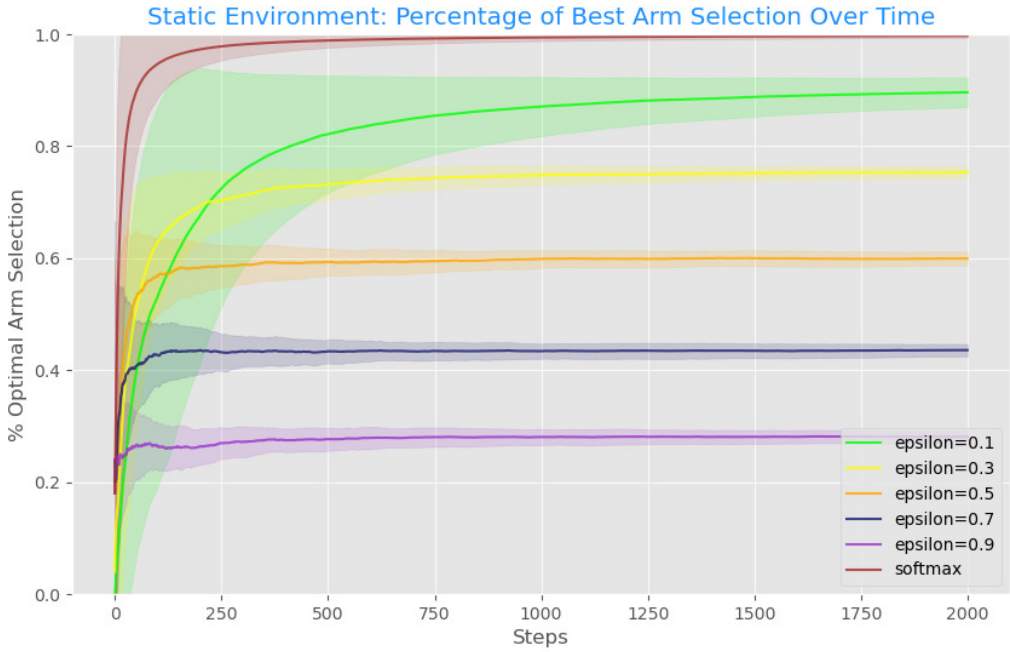

Figure 3: Percentage of best arm selection over time for static environment.

In Figure 3 it is shown that in a stationary environment, lower ε (0.1) quickly converges to selecting the best arm most of the time, giving high optimal-arm percentages. Higher ε values (0.7, 0.9) keep exploring too often, so their optimal-arm selection rate remains low even after many steps. The Softmax policy rapidly concentrates most probability on the best arm while still doing some graded exploration, so it ends up with the highest optimal-arm selection.

0.2 Dynamic Environment

All methods increase cumulative reward, but the slopes separate as the environment changes. The average estimator (green) initially performs well but its curve flattens compared to the others, reflecting slower adaptation and more pulls of suboptimal arms. Moving average (yellow) does better, and the incremental update (orange) achieves the steepest curve, the highest long-term cumulative reward, which means lower regret in a non-stationary bandit.
Figure 5 adds more estimators (different windows, step sizes, ε-greedy) and shows them under higher noise. Despite the noise, methods with larger constant steps and exploration (ε-greedy) tend to track changes better and obtain higher cumulative reward, while methods that average over too much history lag behind because old data dominates their estimates.
At the start, all methods select the best arm rarely, because their value estimates are still very uncertain. The average estimator (green) learns reasonably fast but then degrades as the environment changes, since it keeps all past rewards and therefore reacts slowly to non-stationarity. The moving average (yellow) reacts faster to changes than the plain average, so the percentage of best-arm selections improves more steadily. The incremental update with fixed step size (orange) adapts the quickest and maintains the highest fraction of optimal-arm selections over time, showing that constant-step updates are more appropriate in a dynamic environment. Overall, these graphs support the conclusion that in static environments we want low exploration (or Softmax), while in dynamic environments we need estimators and step sizes that adapt quickly to changes.

- Experimental results show the average estimator decays over time because it calculates the

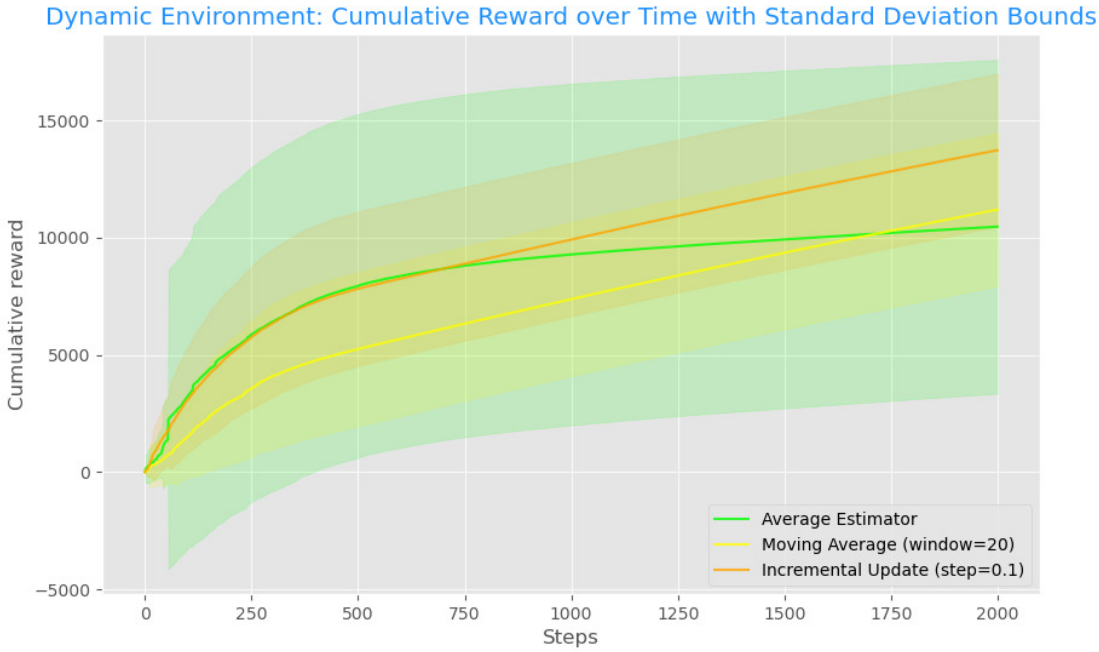

Figure 4: Cumulative reward over time with standard deviation bounds for dynamic environment

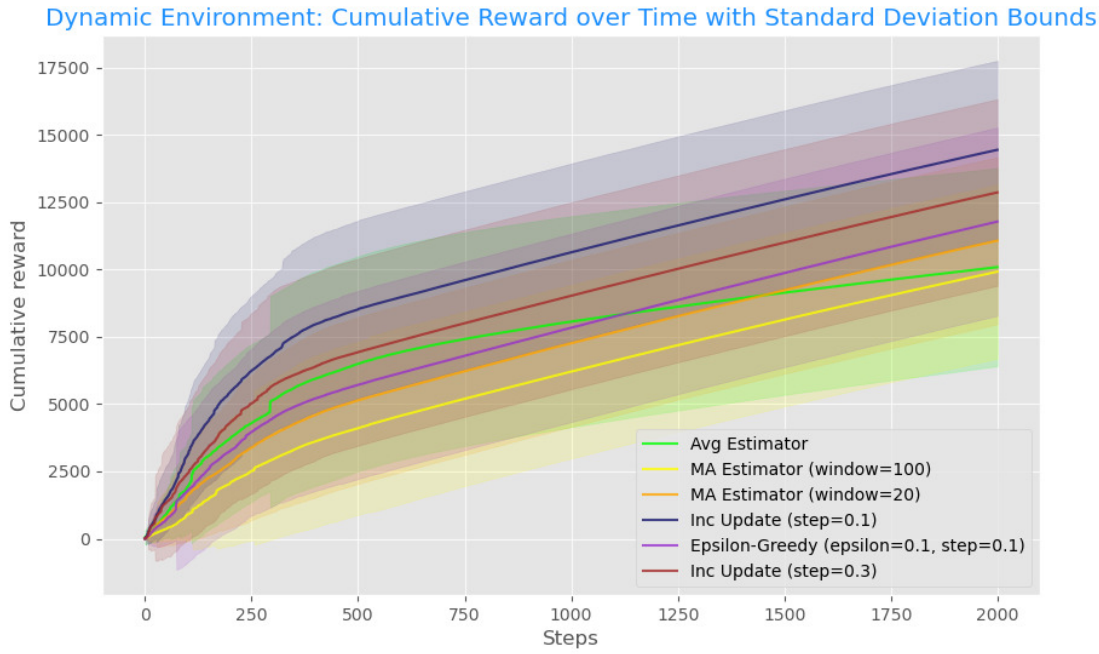

Figure 5: Cumulative reward over time with standard deviation bounds for dynamic environment

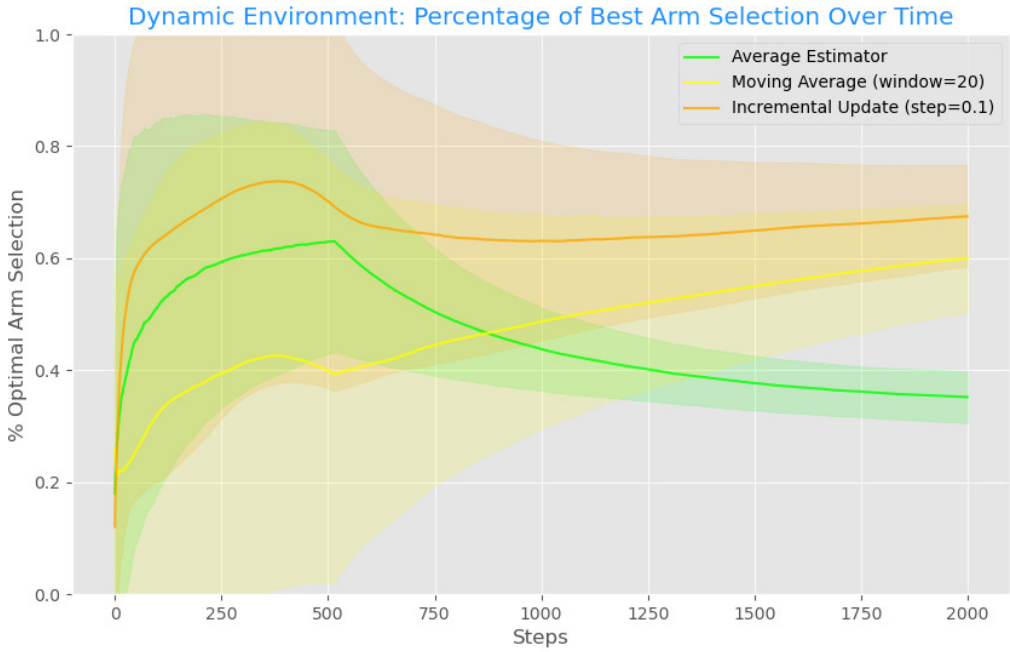

Figure 6: Percentage of Best Arm Selection over time in a Dynamic Environmentt

value with historical memory

- Moving average shows lower efficiency at the start because it has lower information/data to work with, but over time it improves significantly due to the dynamic nature of the estimator
- For the incremental estimator, not only did it start off with relatively high optimal selection but also it performed the most optimally over time. This is because it accumulates data results and it has more accumulative data to work with.
- These results indicate that dynamic estimators are superior to the average estimator, and the incremental estimator is the most optimal (within our timespan). Given more time, there is a potential for the moving average to overtake the incremental estimator, but this cannot be accurately shown from the data.

Discussion

Problem 2

The trade-off between exploration and exploitation is a central challenge in reinforcement learning. To obtain high reward, an agent should prefer actions that have produced good results in the past (exploitation), but to discover such actions it must also try actions that have not been tried often or at all (exploration). (Berger-Tal, Nathan, Meron, & Saltz, 2014). In other words, the agent must exploit what it already knows to gain immediate reward, while also exploring to improve its knowledge and make better action selections in the future. (Sutton & Barto, 2014) In multi-armed bandits, the exploration-exploitation dilemma arises because, at each step, the agent must decide whether to play the arm that has produced the highest average reward so far (exploitation) or to try a different arm to improve its estimate of that arm's true value (exploration). Formally, each arm has an unknown reward distribution, and the agent only learns about these rewards through

repeated interaction, which makes this trade-off unavoidable. The goal of a multi-armed bandit agent is to find an appropriate balance between exploration and exploitation that minimises the regret over a given time horizon. Regret is defined as the difference between the cumulative reward that would have been obtained by always selecting the optimal arm and the cumulative reward obtained by the agent's strategy. A good bandit algorithm aims for regret that grows sublinearly with time, so that the average regret per step converges to zero. (Bubeck & Cesa-Bianchi, 2012) One simple strategy to address this trade-off is the epsilon-greedy method. In epsilon-greedy, at each step the agent chooses a random arm with probability ε (exploration) and chooses the arm with the highest current estimated value with probability $1 - \varepsilon$ (exploitation). The parameter ε therefore controls the balance between exploration and exploitation: larger values of ε encourage more exploration, while smaller values favour exploiting the best-known arm. (Sutton & Barto, 2014)

Problem 3

The true value of an action is the average reward that was received after that action was taken. The difference between methods lies in how they use that uncertainty to balance exploration and exploitation. In our project we looked at some basic ways to estimate that value and used it to decide what to do next. In terms of static and dynamic environments, we mean the stationary and non-stationary environments.
Key Action-Value methods used:

1. Sample-Average Estimator

This method estimates each arm's value by calculating the mean of all rewards received from that arm. It was tested with epsilon values of 0.2, 0.5, and 0.7.... In static environments it works well, especially when consistent with small epsilon valued. The agent does not need much exploration once it learns the arm values. However, in dynamic environments this estimator becomes a problem because it gives equal weight to old and new rewards, so it cannot adapt quic wh the environment changes.

2. Moving Average Estimator

Instead of using all history, this method only considers the last N rewards (windows of 100 and 20 were used). At the beginning it performs worse than sample-average because it has less data. But in dynamic environments it improves over time since it forgets old rewards and reacts better to changes. A window of 20 showed slightly better results than a window of 100 in most cases.

3. Incremental Update Rule

Uses the formula $Q_{n+1} = Q_n + \alpha[R_n - Q_n]$ with a fixed step size α, which makes recent rewards more important than old ones. This was the best estimator for dynamic environments. It adapted the fastest to reward distribution changes and kept the highest percentage of optimal arm selections over time. It also started well from the beginning, not only at the end. The team notes that moving average could eventually overcome it given more time, but this was not confirmed by the data.

4. ε-Greedy Strategy

The agent exploits the best known arm with probability $1 - \varepsilon$ and explores randomly with probability ε. Values of 0.2, 0.5 and 0.7 were compared. In static environments, low epsilon (0.1-0.3) was clearly better because excessive exploration hurts performance when arm values do not change. In dynamic environments, moderate exploration was more useful because the

agent needs to detect when the best arm changes. Results varied between seeds, which shows the stochastic nature of the problem.

5. Softmax Strategy

Instead of a binary explore/exploit decision, Softmax assigns probabilities to each arm based on learned preferences, updated by gradient ascent.

Subtitle: Considerations on Action Value Methods

Softmax was the best strategy overall for static environments. It concentrated probability on the best arm faster than epsilon-greedy while still doing some exploration. It achieved the highest optimal-arm selection rate among all strategies tested. In general, static environments need low exploration and stable estimators, while dynamic environments need methods that can forget old data and keep exploring. The incremental update is the most suitable estimator for non-stationary settings.

Conclusions

We found that the SoftMax strategy was the best performing one on our stationary setup. We found that, when faced with responsive bandits, the best estimator was incremental update, followed by moving average with a window of 20. Total average estimators are very unreliable when dealing with non-static bandits (what we defined as a dynamic environment).

References

Antonacopoulos, A., Chaudhuri, S., Chellappa, R., Liu, C.-L., Bhattacharya, S., & Pal, U. (Eds.). (2025). *Pattern recognition: 27th international conference, ICPR 2024, kolkata, india, december 1–5, 2024, proceedings, part IX* (Vol. 15309). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-78189-6

Berger-Tal, O., Nathan, J., Meron, E., & Saltz, D. (2014). The exploration-exploitation dilemma: A multidisciplinary framework [Publisher: Public Library of Science]. *PLOS ONE*, *9*(4), e95693. https://doi.org/10.1371/journal.pone.0095693

Bubeck, S., & Cesa-Bianchi, N. (2012, November 3). Regret analysis of stochastic and nonstochastic multi-armed bandit problems. https://doi.org/10.48550/arXiv.1204.5721

Nandhini, S. K., & Pandian, J. A. (2025). Fractal dimension analysis for improved decision-making in non-stationary multi-arm bandit problems: A machine learning approach. *The European Physical Journal Special Topics*. https://doi.org/10.1140/epjs/s11734-025-01581-4

Sutton, R. S., & Barto, A. G. (n.d.). Reinforcement learning: An introduction.

Wang, H., Zariphopoulou, T., & Zhou, X. (2019, February 13). Exploration versus exploitation in reinforcement learning: A stochastic control approach. https://doi.org/10.48550/arXiv.1812.01552